Executive Report

Vyoma E-Learning Data Warehouse

Executive Summary

This project is a complete data warehouse and reporting pipeline for Vyoma Samskrta Pathasala's Edmingle LMS data. It collects learner, enrollment, course, batch, attendance, session, assessment, certificate, and announcement data from Edmingle, stores it in PostgreSQL, and prepares clean reporting datasets for Power BI.

The system follows a Bronze/Silver/Gold medallion architecture:

- Bronze keeps raw source data for auditability and recovery.
- Silver stores cleaned, typed, deduplicated operational records.
- Gold exposes reporting-ready SQL views for Power BI dashboards.

The repository includes database schema files, ingestion services, scheduled API scripts, historical CSV migration scripts, operational tools, test suites, and documentation. It is designed for both real-time webhook ingestion and scheduled API pulls.

GitHub Repository

Current repository inventory:

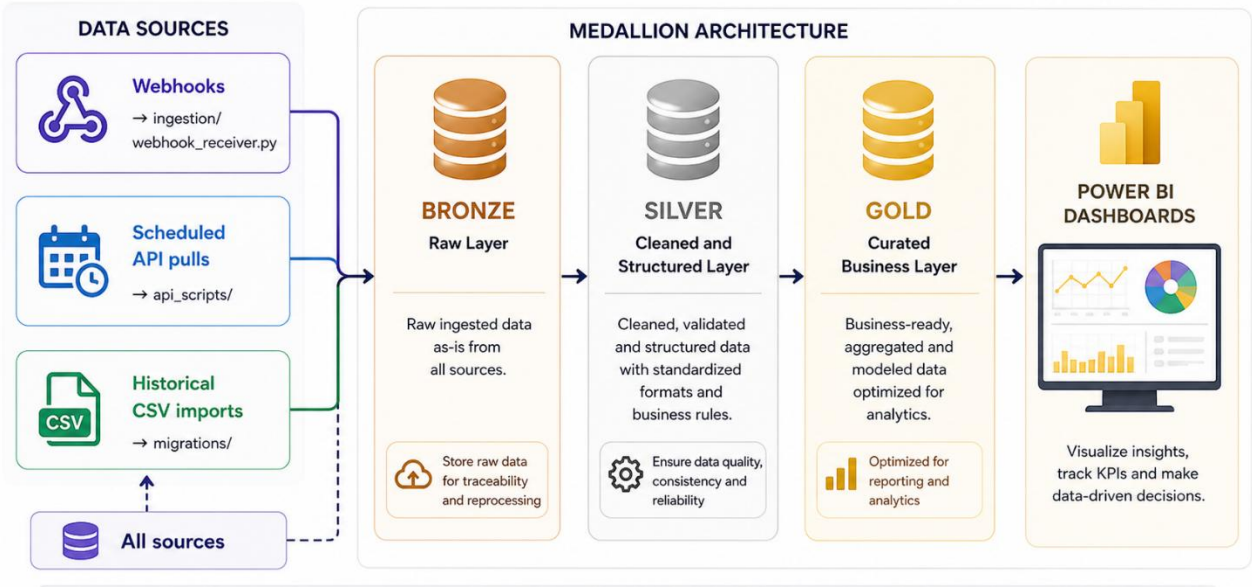
Area	Count / Contents
Total tracked project files	81
Python scripts/tests	19
SQL schema/view files	26
Main documentation folder	docs/
Data source folders	api_scripts/, ingestion/, migrations/, csv_files/, pdf/
Database layers	database/bronze, database/silver, database/gold
Operational tools	tools/
Test suites	tests/

Data Sources

Source	Method	Data Captured	Status
Edmingle webhooks	Real-time HTTP POST	Users, transactions, sessions, assessments, course completions, announcements, certificates	Built
Edmingle attendance API	Scheduled pull, report_type=55	Per-student class attendance, summarized into batch/date attendance	Built
Edmingle course catalogue API	Scheduled pull	Course bundle metadata and Vyoma classification fields	Built
Edmingle master batch API	Scheduled pull	Batch names, dates, tutor, enrollment count	Built
Historical student CSV	One-time migration	Student profile history	Built
Historical enrollment CSV	One-time migration	Historical course enrollments	Built
Course lifecycle CSV	One-time migration	Operational course lifecycle metrics	Built

Architecture Overview

The data flow is:



Key design strengths:

- Raw source data is preserved before transformation.
- Silver transformations are idempotent through UPSERT logic.
- Failed webhook handling prevents silent data loss.
- Disk fallback queue protects against temporary PostgreSQL outages.
- Gold views avoid duplicated reporting tables while staying fresh.
- The system can reprocess Bronze into Silver when transformation logic changes.

Database Coverage

Bronze Layer

Bronze stores raw or near-raw data exactly as received:

- bronze.webhook_events
- bronze.failed_events
- bronze.attendance_raw
- bronze.course_catalogue_raw
- bronze.course_batches_raw
- bronze.course_lifecycle_raw
- bronze.studentexport_raw
- bronze.studentcoursesenrolled_raw
- bronze.unresolved_students_raw

Silver Layer

Silver stores cleaned and deduplicated business entities:

- silver.users
- silver.transactions
- silver.sessions
- silver.assessments
- silver.courses
- silver.announcements
- silver.certificates
- silver.course_metadata
- silver.course_batches
- silver.course_lifecycle
- silver.course_master
- silver.class_attendance

Gold Layer

Gold exposes reporting views:

- Course and enrollment views: gold.courses, gold.learners, gold.course_type, gold.launch_type, gold.subject, gold.learning_model, gold.term, gold.funnel, gold.sss_domain
- Attendance views: gold.batch_attendance, gold.bundle_attendance, gold.attendance_by_year, gold.first_vs_last_class

Current Reporting Value

The project currently supports executive and operational dashboards for:

- Total enrollments by course and batch.
- Live vs historical enrollment source split.
- New vs returning learner trends.
- Course distribution by subject, SSS category, adhyayanam category, term, and funnel position.
- Attendance percentage by batch, course, and year.
- First-class vs last-class attendance drop-off.
- Course lifecycle performance including certification, ratings, and first/last class metrics.
- Teacher and batch-level course operations through batch metadata.

Reliability and Controls

The project already includes several important reliability controls:

- HMAC signature verification for webhook authenticity.
- Bronze audit logging before Silver routing.
- Duplicate event protection through unique keys and UPSERTs.
- Fallback disk queue for events received during PostgreSQL outages.
- Reprocessing tools for Bronze-to-Silver recovery.
- Failed-event storage for malformed or unprocessable requests.
- End-to-end tests for event routing, deduplication, concurrency, and outage recovery.

Test coverage:

Test Suite	Coverage
tests/test_pipeline_e2e.py	38 full webhook pipeline tests
tests/test_db_unavailability.py	PostgreSQL outage and fallback recovery
tests/test_all_events.py	Coverage for all supported webhook event types

Key Risks and Gaps

Risk / Gap	Business Impact	Recommended Action
Credentials are hardcoded in script-level CONFIG blocks	Higher risk of accidental credential exposure and inconsistent production configuration	Move secrets to environment variables or a single local config file excluded from git
Historical CSV files contain PII and are intentionally gitignored	Data must be handled carefully outside version control	Maintain documented storage, access, and retention procedures

Risk / Gap	Business Impact	Recommended Action
bronze.unresolved_students_raw contains unmatched historical students	Some historical learner records cannot join to current user/enrollment reporting	Build resolver using Edmingle student list API or another user export with IDs
Attendance backfill is still growing	Attendance dashboards may be incomplete until backfill finishes	Track completion by date range and batch coverage
Some Edmingle export APIs are documented but not implemented	Engagement, material-view, and quiz/exercise analytics are not yet available	Prioritize API data export types 5, 8, and 10 if engagement reporting is required
Production deployment depends on manual setup	Operational consistency depends on runbook discipline	Add automated deployment checklist or scripts for VPS setup
Gold views are non-materialized	Large Power BI queries may slow down as data grows	Monitor query performance; materialize selected views only if needed

Recommended Next Actions

1. Add a secure centralized configuration approach for DB credentials, API keys, and webhook secrets.
2. Build an unresolved-student matching process using the Edmingle student API or a richer user export.
3. Add scheduled automation for daily attendance and course pipeline runs.
4. Add monitoring for daily row counts, failed webhook count, fallback queue presence, and stale Power BI data.